

Assignment 5 :

- Github Link :
- README.md Link :

Question 1 : Mux 2 to 1

Verilog code :

```
module mux_2_to_1_behavioral (  
    input wire a,  
    input wire b,  
    input wire sel,  
    output reg y  
);  
    always @(sel, a, b) begin  
        if (sel)  
            y = a;  
        else  
            y = b;  
        end  
    endmodule
```

Testbench code :

```
`include "mux_2_to_1_behavioral.v"  
module mux_2_to_1_behavioral_tb;  
    reg a, b, sel;  
    wire y;  
    mux_2_to_1_behavioral uut (.a(a), .b(b), .sel(sel), .y(y));  
    initial begin  
        $dumpfile("mux_2_to_1_behavioral_tb.vcd");
```

```

$dumpvars(0, mux_2_to_1_behavioral_tb);

$monitor("Time = %0t | a=%b b=%b sel=%b | y=%b", $time, a, b, sel, y);

#0  a = 0; b = 0; sel = 0;

#10 a = 0; b = 1; sel = 0;

#10 a = 1; b = 0; sel = 0;

#10 a = 1; b = 1; sel = 0;

#10 a = 0; b = 0; sel = 1;

#10 a = 0; b = 1; sel = 1;

#10 a = 1; b = 0; sel = 1;

#10 a = 1; b = 1; sel = 1;

#10 $finish;

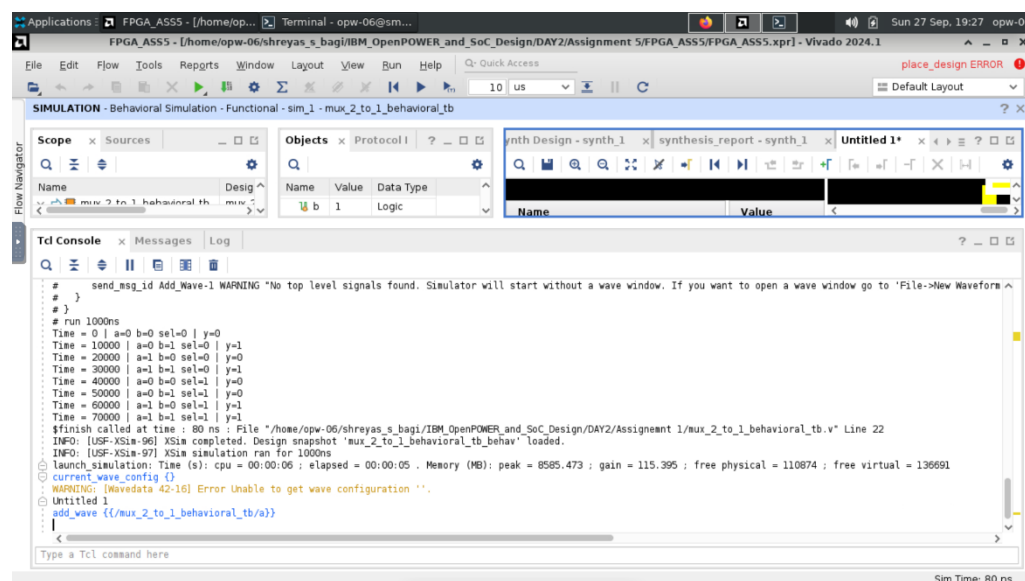
end

endmodule

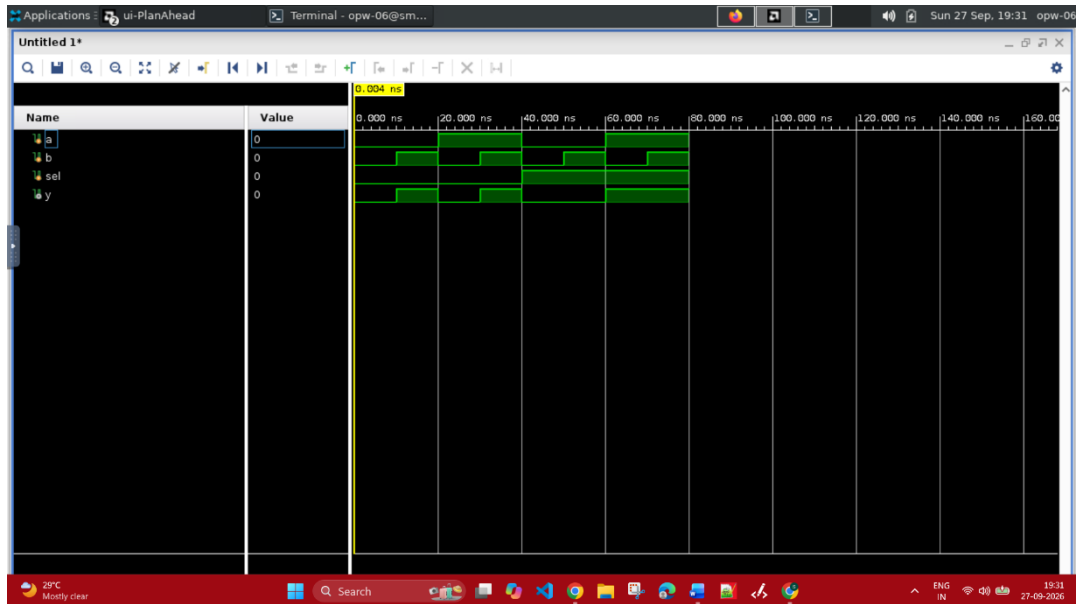
```

Compilation and Simulation Snapshot :

From the screenshot we observe for every 10 nanoseconds there is change in the output variable y. The selection when sel = 0 then we select the variable b and when sel = 1 then we select the variable a.



Waveform Window Snapshot : The same above what is observed in the log file even captured in the waveform window also.



Question 2 :

Design a self-checking testbench for a 4-bit Magnitude Comparator and simulate using Vivado.

Verilog Code :

```
module comparator_4bit (
    input [3:0] a,
    input [3:0] b,
    output reg a_greater,
    output reg b_greater,
    output reg equal
);
    always @(*) begin
        a_greater = 1'b0;
        b_greater = 1'b0;
        equal = 1'b0;

        if (a[3] != b[3]) begin
            a_greater = a[3];
            b_greater = b[3];
        end else if (a[2] != b[2]) begin
            a_greater = a[2];
            b_greater = b[2];
        end else if (a[1] != b[1]) begin
            a_greater = a[1];
            b_greater = b[1];
        end else if (a[0] != b[0]) begin
            a_greater = a[0];
            b_greater = b[0];
        end else begin
```

```

        equal = 1'b1;
    end
end
endmodule

```

Testbench Code :

```
`timescale 1ns/1ps
```

```
module comparator_4bit_tb;
```

```
    reg [3:0] a;
```

```
    reg [3:0] b;
```

```
    wire    a_greater;
```

```
    wire    b_greater;
```

```
    wire    equal;
```

```
    integer input_value_a;
```

```
    integer input_value_b;
```

```
    integer random_value_a;
```

```
    integer random_value_b;
```

```
    integer random_test_index;
```

```
    integer random_seed;
```

```
    integer test_count;
```

```
    integer failure_count;
```

```
    comparator_4bit dut (
```

```
        .a(a),
```

```
        .b(b),
```

```
        .a_greater(a_greater),
```

```
.b_greater(b_greater),  
.equal(equal)  
);
```

```
task automatic check_case;
```

```
    input [3:0] test_a;
```

```
    input [3:0] test_b;
```

```
    begin
```

```
        a = test_a;
```

```
        b = test_b;
```

```
        #1;
```

```
        test_count = test_count + 1;
```

```
        if (test_a > test_b) begin
```

```
            $display("Test %0d: A=%0d, B=%0d -> A > B | result: a_greater=%b  
b_greater=%b equal=%b",
```

```
                test_count, test_a, test_b, a_greater, b_greater, equal);
```

```
            if ({a_greater, b_greater, equal} !== 3'b100) begin
```

```
                $display("FAIL: expected A > B, got %b", {a_greater, b_greater, equal});
```

```
                failure_count = failure_count + 1;
```

```
            end
```

```
        end else if (test_a < test_b) begin
```

```
            $display("Test %0d: A=%0d, B=%0d -> A < B | result: a_greater=%b  
b_greater=%b equal=%b",
```

```
                test_count, test_a, test_b, a_greater, b_greater, equal);
```

```
            if ({a_greater, b_greater, equal} !== 3'b010) begin
```

```
                $display("FAIL: expected A < B, got %b", {a_greater, b_greater, equal});
```

```
                failure_count = failure_count + 1;
```

```
            end
```

```

        end else begin

            $display("Test %0d: A=%0d, B=%0d -> A == B | result: a_greater=%b
b_greater=%b equal=%b",

                test_count, test_a, test_b, a_greater, b_greater, equal);

            if ({a_greater, b_greater, equal} !== 3'b001) begin

                $display("FAIL: expected A == B, got %b", {a_greater, b_greater, equal});

                failure_count = failure_count + 1;

            end

        end

    end

end

endtask

```

```

initial begin

    $dumpfile("4bit_comparator_tb.vcd");

    $dumpvars(0, comparator_4bit_tb);

    test_count = 0;

    failure_count = 0;

    random_seed = 32'h1ACE_B00C;

    $display("Starting 20 deterministic random comparator tests...");

    for (random_test_index = 0; random_test_index < 20; random_test_index =
random_test_index + 1) begin

        random_value_a = $random(random_seed) & 4'hf;

        random_value_b = $random(random_seed) & 4'hf;

        check_case(random_value_a, random_value_b);

    end

    $display("Starting 256-case exhaustive comparator test...");

```

```

for (input_value_a = 0; input_value_a < 16; input_value_a = input_value_a + 1) begin
    for (input_value_b = 0; input_value_b < 16; input_value_b = input_value_b + 1)
begin
    check_case(input_value_a, input_value_b);

    end
end

if (failure_count == 0) begin

    $display("PASS: %0d random and exhaustive comparator tests completed
successfully.", test_count);

    end else begin

        $display("FAIL: %0d of %0d comparator tests failed.", failure_count, test_count);

        $fatal(1);

    end

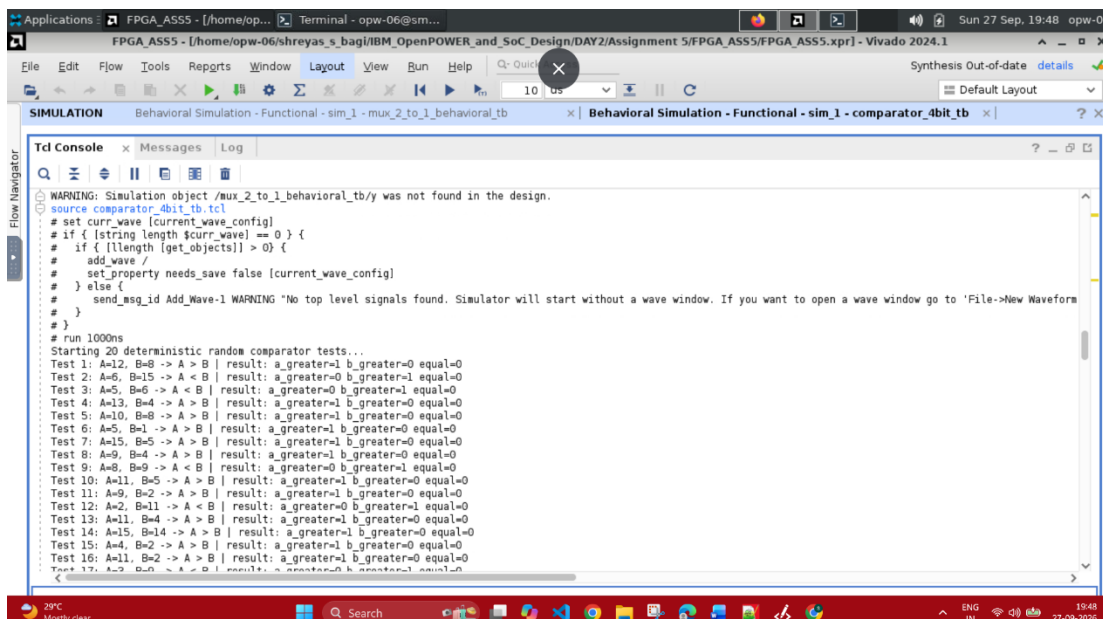
$finish;

end

endmodule

```

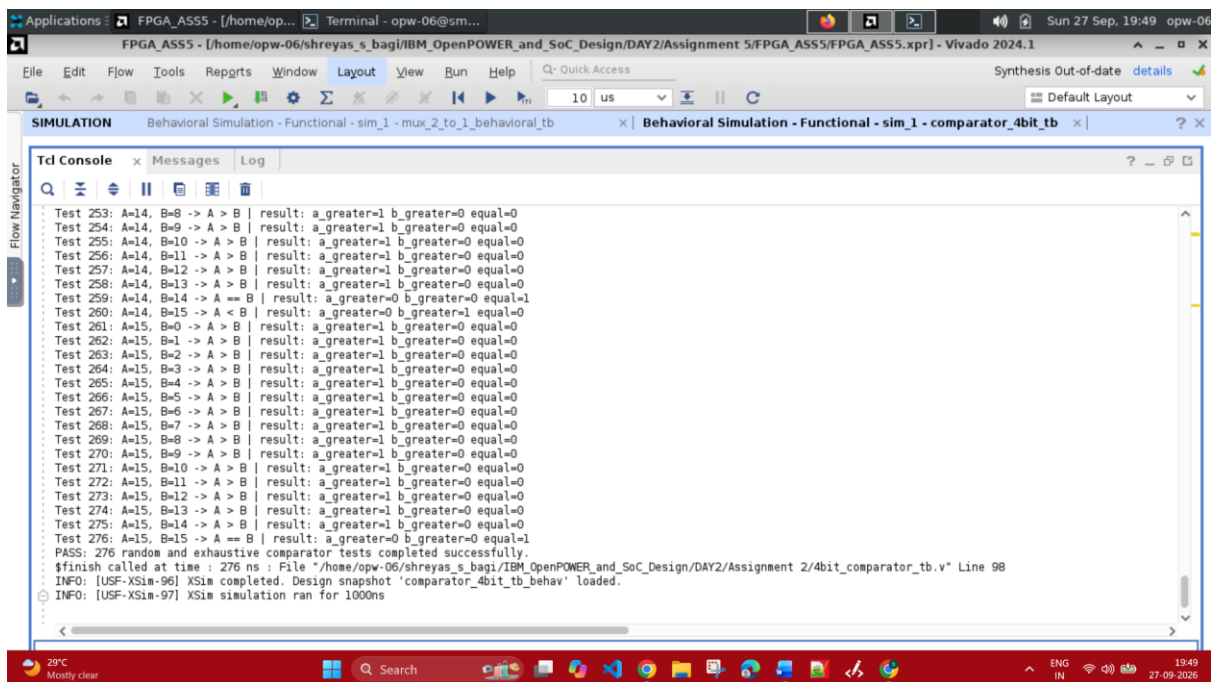
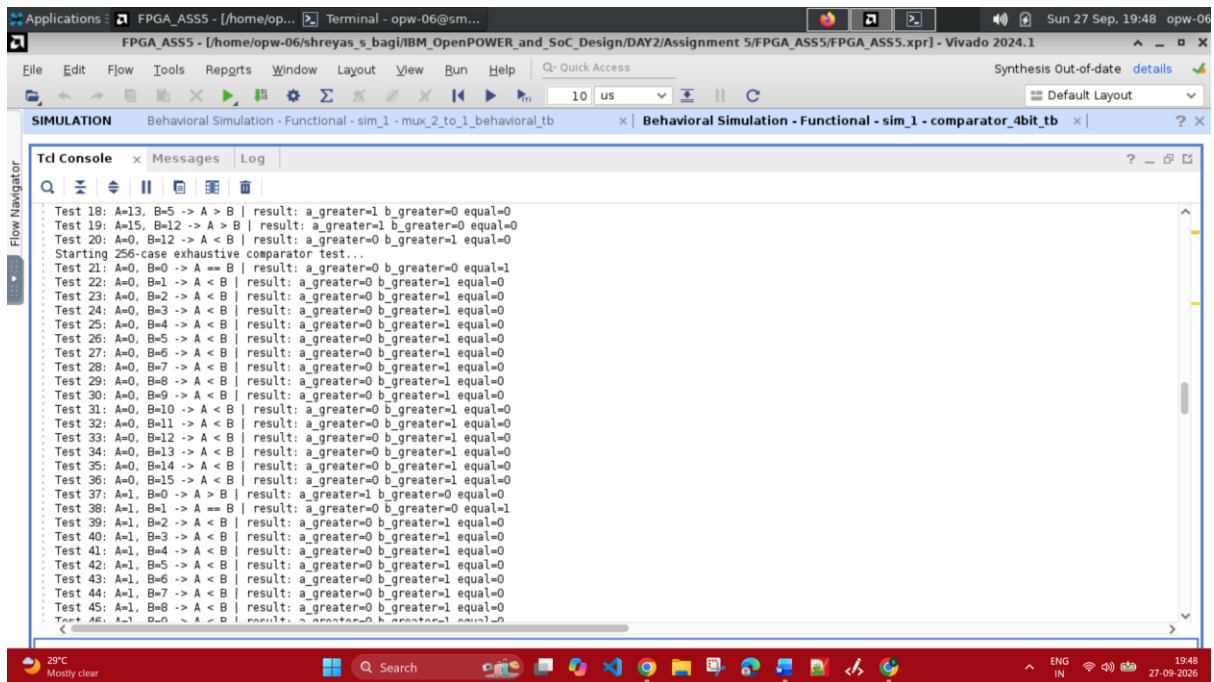
Compilation and Simulation :



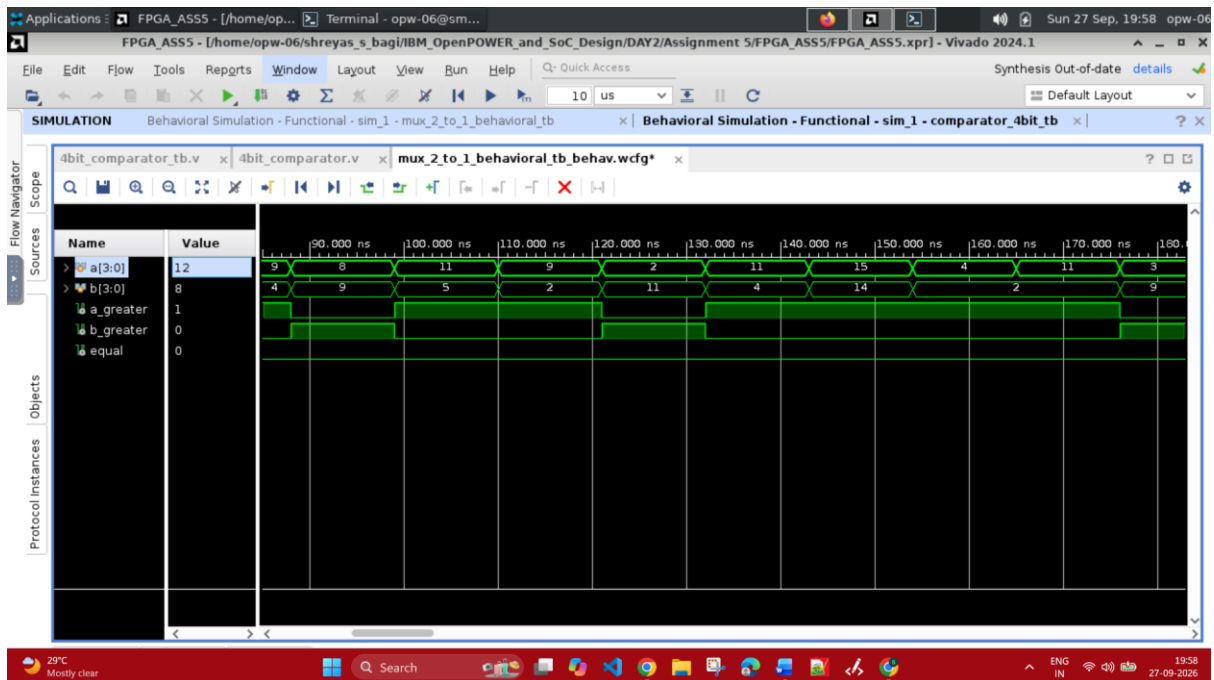
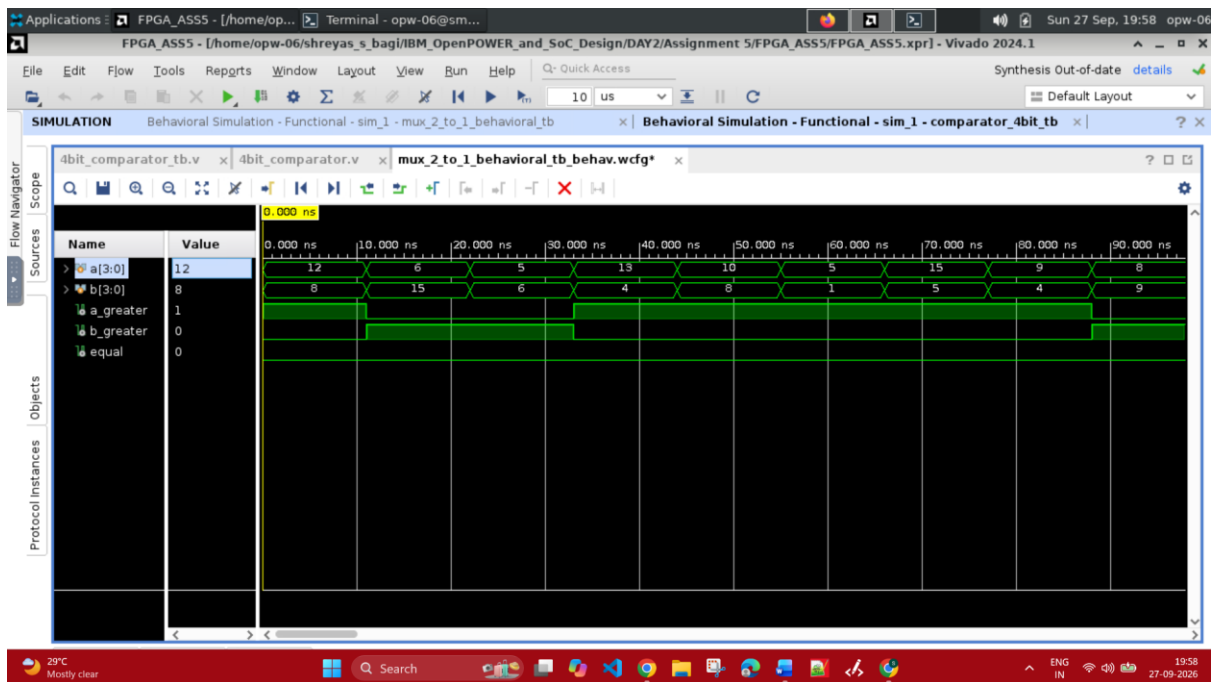
```

Tcl Console x Messages Log
WARNING: Simulation object /mux_2_to_1_behavioral_tb/y was not found in the design.
source comparator_4bit_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
#   }
# }
# run 1000ns
Starting 20 deterministic random comparator tests...
Test 1: A=12, B=8 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 2: A=6, B=15 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 3: A=5, B=6 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 4: A=13, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 5: A=10, B=8 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 6: A=5, B=1 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 7: A=15, B=5 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 8: A=9, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 9: A=8, B=9 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 10: A=11, B=5 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 11: A=9, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 12: A=2, B=11 -> A < B | result: a_greater=0 b_greater=1 equal=0
Test 13: A=11, B=4 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 14: A=15, B=14 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 15: A=4, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 16: A=11, B=2 -> A > B | result: a_greater=1 b_greater=0 equal=0
Test 17: A=3, B=0 -> A > B | result: a_greater=1 b_greater=0 equal=0

```



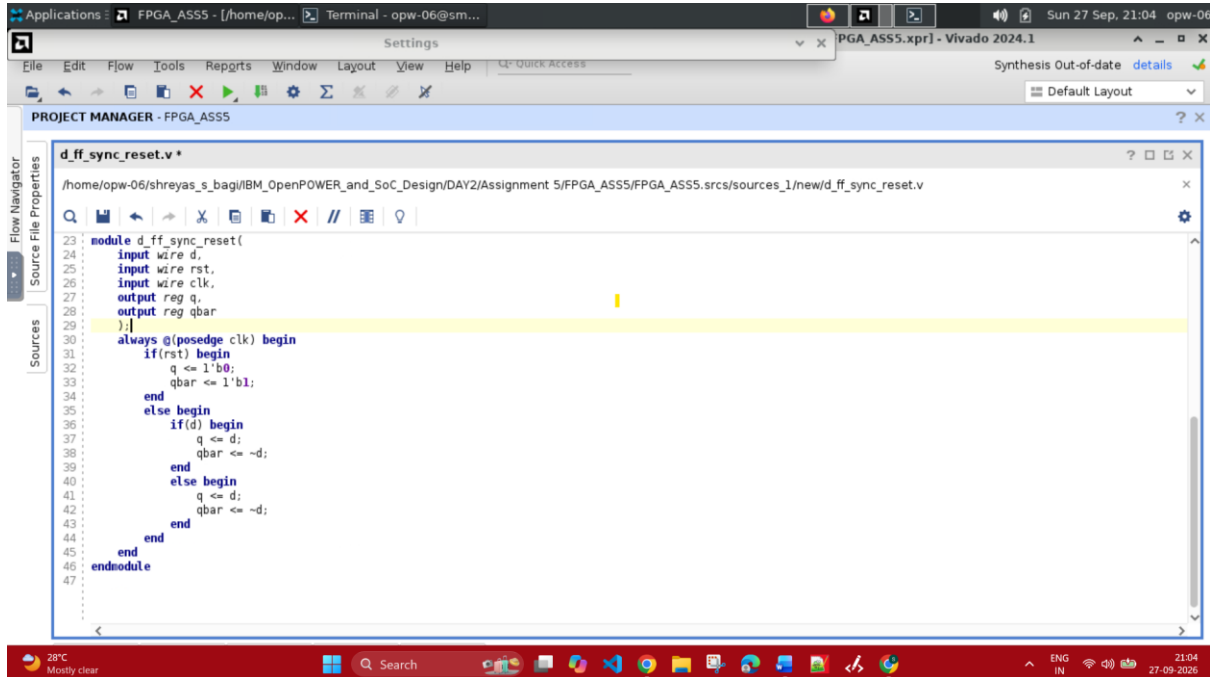
Simulation Waveform :



Question 3 :

Write a complete testbench for a D Flip-Flop with synchronous reset and simulate in Vivado.

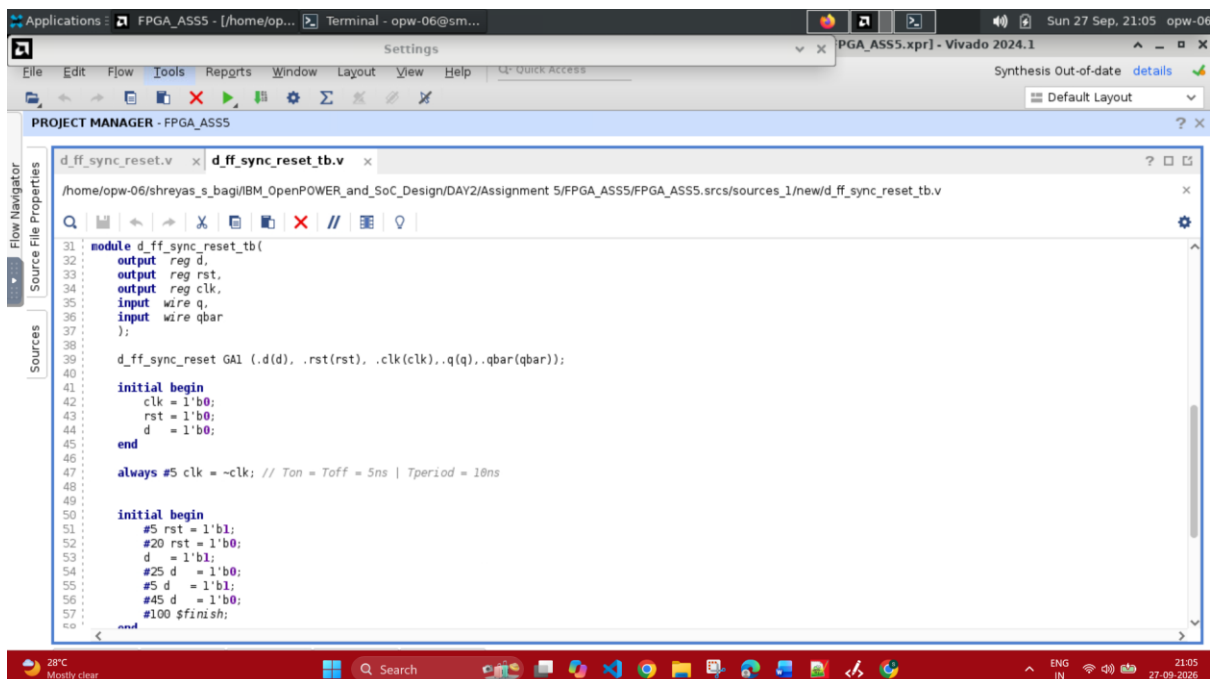
Verilog code :



The screenshot shows the Vivado IDE interface. The main window displays the Verilog code for a module named `d_ff_sync_reset.v`. The code is as follows:

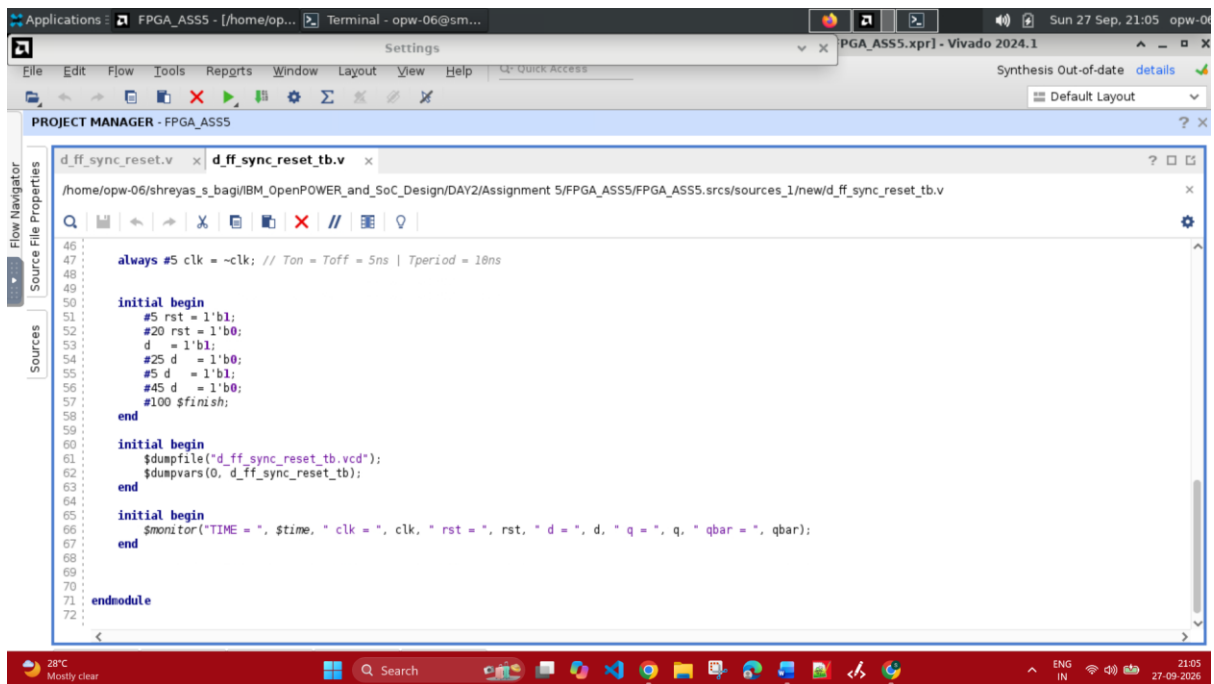
```
23: module d_ff_sync_reset(  
24:     input wire d,  
25:     input wire rst,  
26:     input wire clk,  
27:     output reg q,  
28:     output reg qbar  
29: );  
30: always @(posedge clk) begin  
31:     if(rst) begin  
32:         q <= 1'b0;  
33:         qbar <= 1'b1;  
34:     end  
35:     else begin  
36:         if(d) begin  
37:             q <= d;  
38:             qbar <= ~d;  
39:         end  
40:         else begin  
41:             q <= d;  
42:             qbar <= ~d;  
43:         end  
44:     end  
45: end  
46: endmodule  
47:
```

Testbench code :

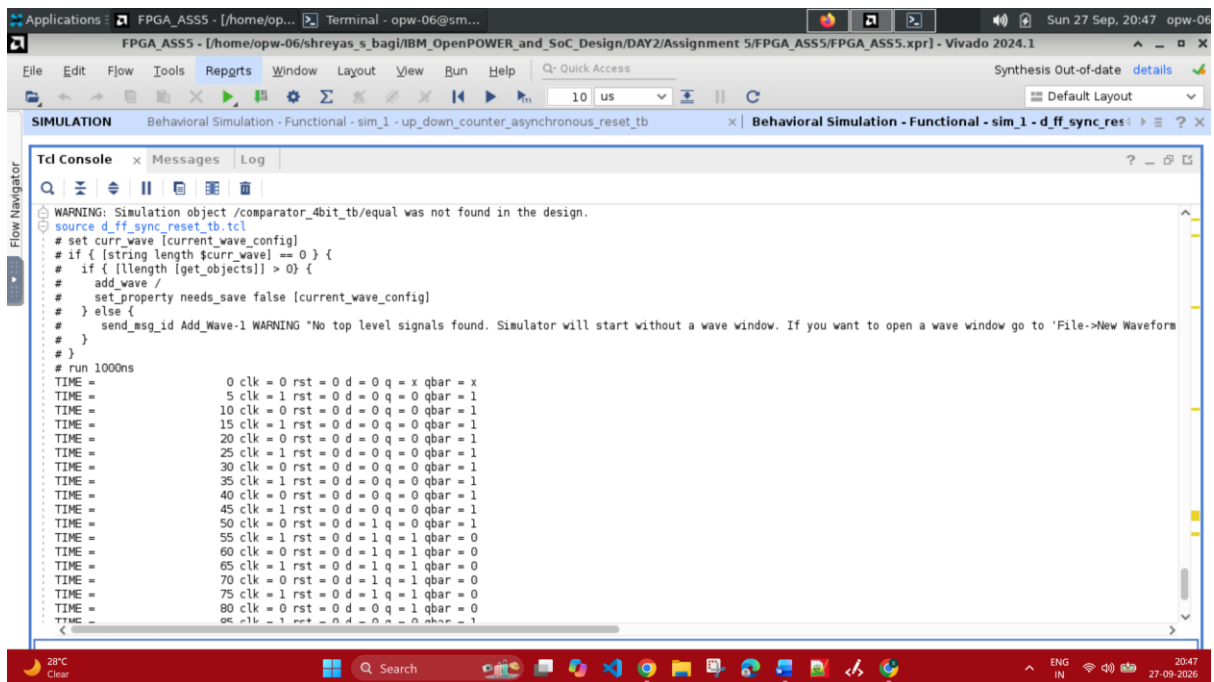


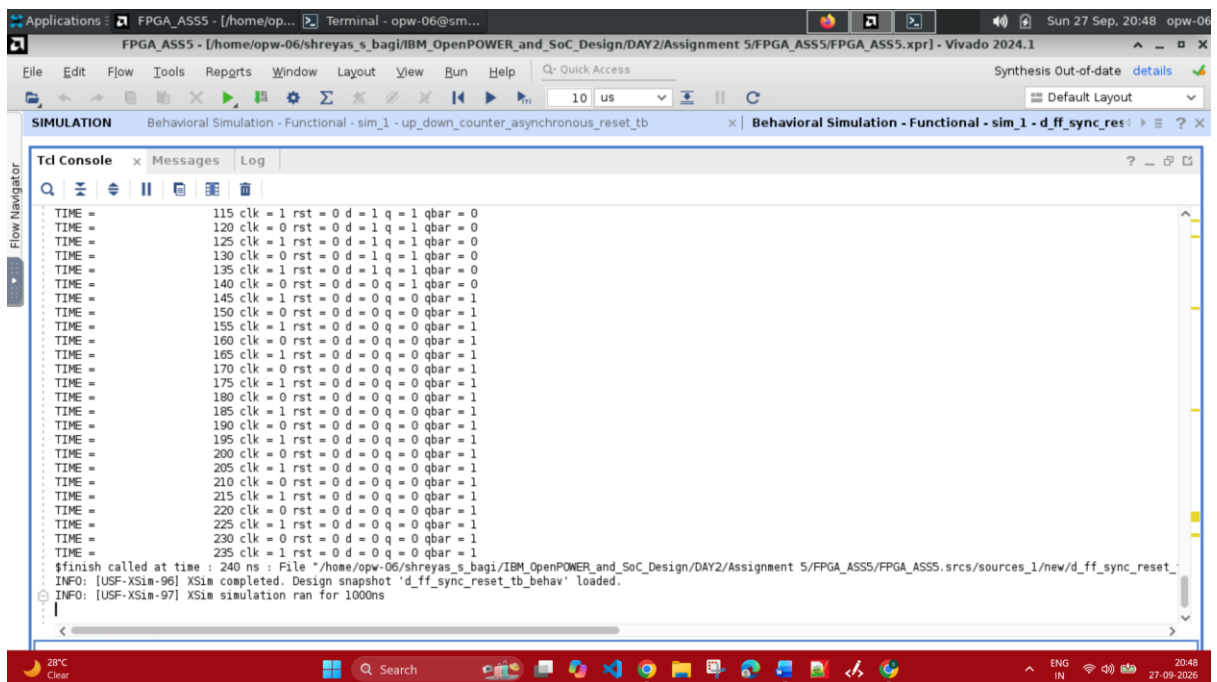
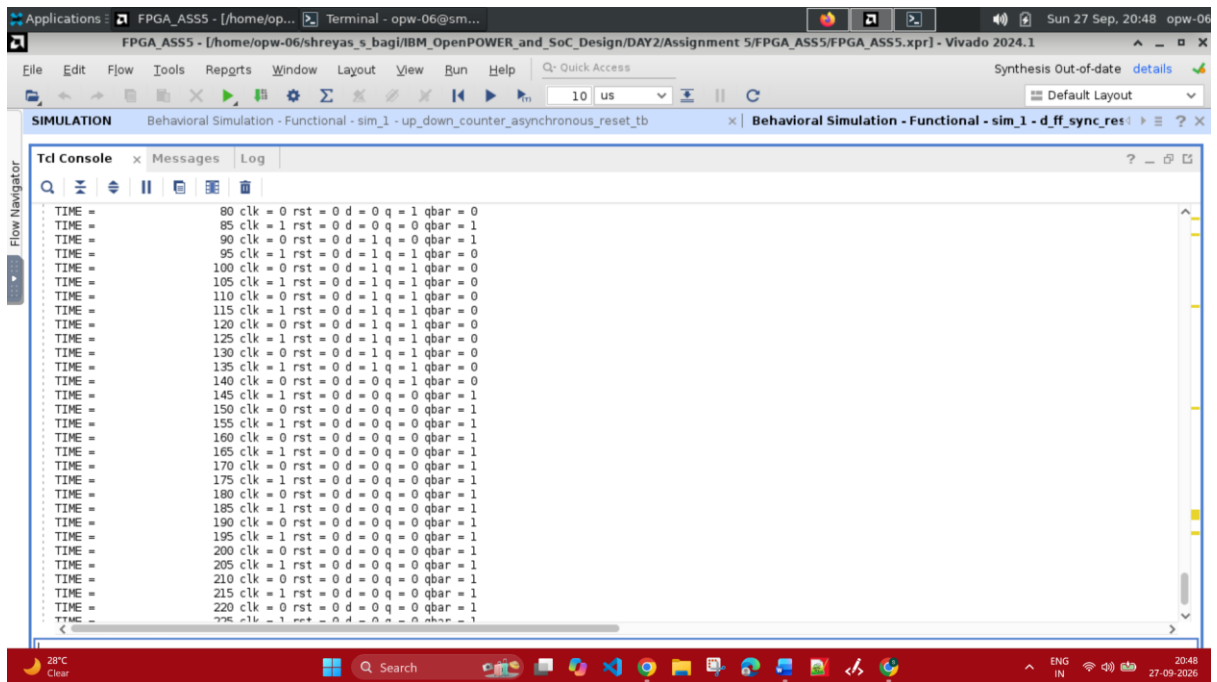
The screenshot shows the Vivado IDE interface with the testbench code for the `d_ff_sync_reset` module. The testbench code is as follows:

```
31: module d_ff_sync_reset_tb(  
32:     output reg d,  
33:     output reg rst,  
34:     output reg clk,  
35:     input wire q,  
36:     input wire qbar  
37: );  
38: d_ff_sync_reset GA1 (.d(d), .rst(rst), .clk(clk), .q(q), .qbar(qbar));  
39:  
40: initial begin  
41:     clk = 1'b0;  
42:     rst = 1'b0;  
43:     d = 1'b0;  
44: end  
45:  
46: always #5 clk = ~clk; // Ton = Toff = 5ns | Tperiod = 10ns  
47:  
48: initial begin  
49:     #5 rst = 1'b1;  
50:     #20 rst = 1'b0;  
51:     d = 1'b1;  
52:     #25 d = 1'b0;  
53:     #5 d = 1'b1;  
54:     #45 d = 1'b0;  
55:     #100 $finish;  
56: end  
57:  
58: endmodule  
59:
```



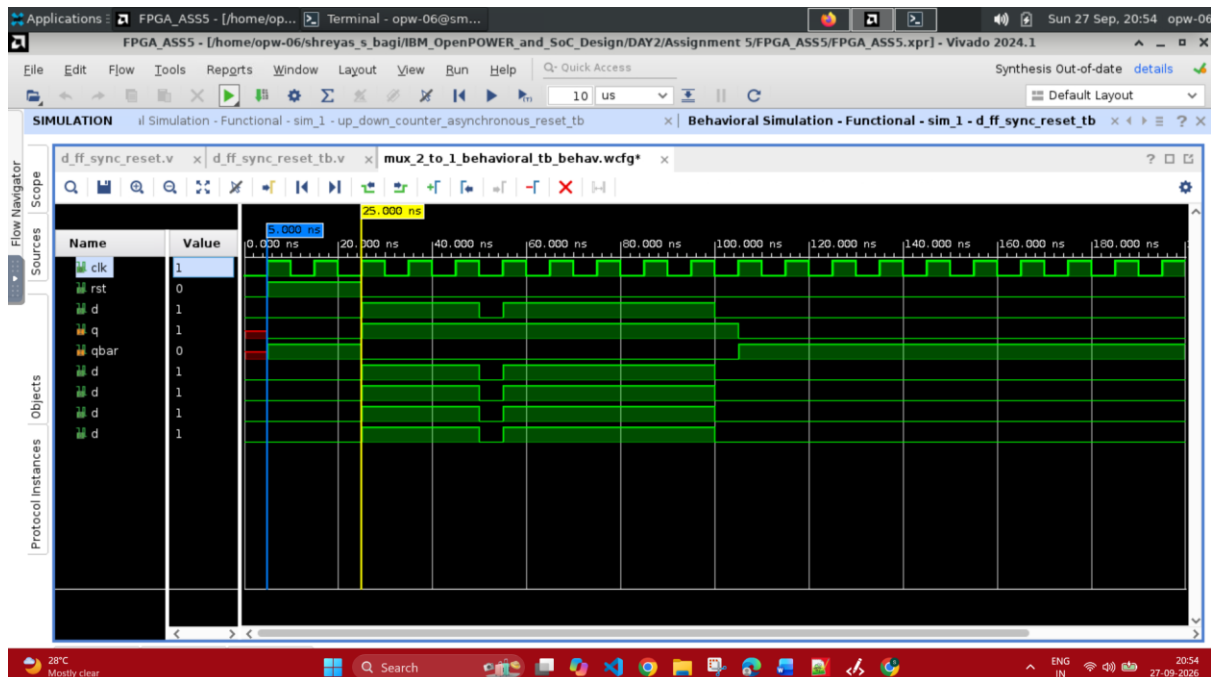
Compilation and Simulation Output :



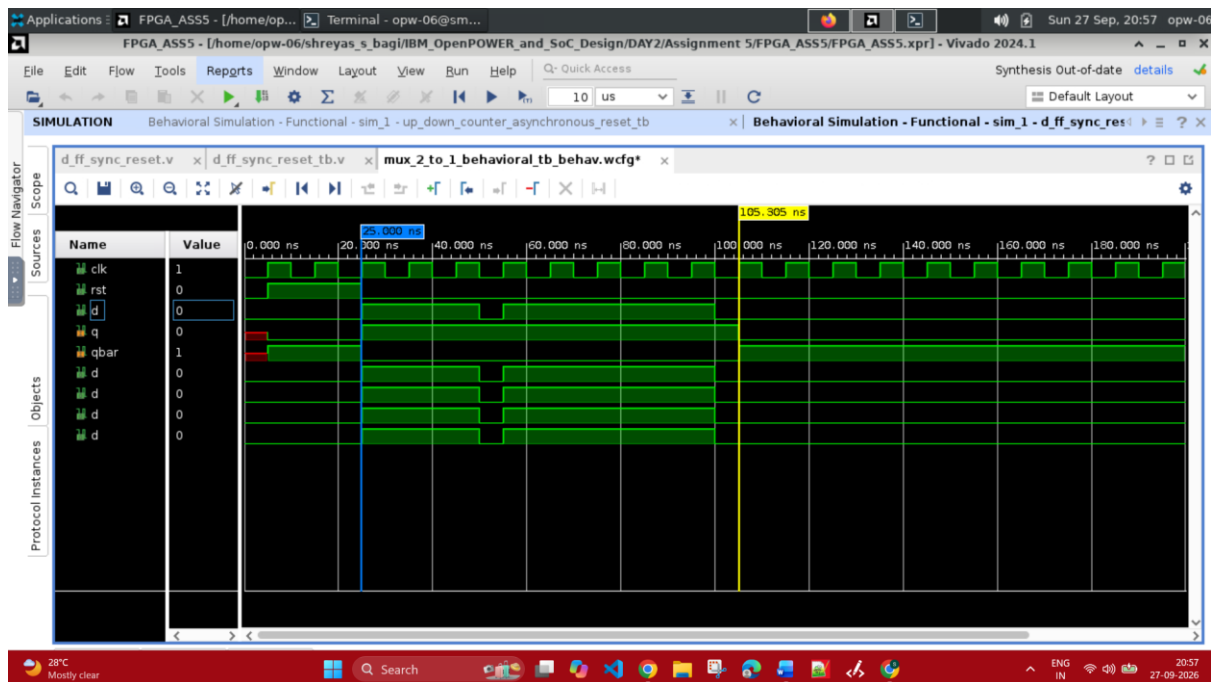


Simulation Waveform :

First two clk edges rst is applied then we see the D Flip Flop is Synchronous reset.



At clk = 25ns to 100ns we observe that d = 1 on every posedge of the clock and stuff hence we see q = 1 and qbar = 0. At clk = 105ns we observe that d = 0 on every posedge of the clock and stuff hence we see q = 0 and qbar = 1.



Question 4 :

Write and simulate a testbench for the 4-bit Up/Down Counter (Assignment 04) in Vivado.

Verilog Code :

```
module up_down_counter_asynchronous_reset (
    input      clk,
    input      reset,
    input      up,
    output reg [3:0] count
);
    always @(posedge clk or posedge reset) begin
        if (reset)
            count <= 4'b0000;
        else if (up)
            count <= count + 4'b0001;
        else
            count <= count - 4'b0001;
        end
    endmodule
```

Testbench Code :

```
/* Testbench for the asynchronous-reset up/down counter. */
`timescale 1ns/1ps
module up_down_counter_asynchronous_reset_tb;
    reg clk;
    reg reset;
    reg up;
    wire [3:0] count;
```

```
integer test_count;
```

```
integer failure_count;
```

```
integer value;
```

```
reg [87:0] mode;
```

```
up_down_counter_asynchronous_reset dut (
```

```
    .clk(clk),
```

```
    .reset(reset),
```

```
    .up(up),
```

```
    .count(count)
```

```
);
```

```
always #5 clk = ~clk;
```

```
task check_count;
```

```
    input [3:0] expected_count;
```

```
    begin
```

```
        #1;
```

```
        test_count = test_count + 1;
```

```
        if (reset)
```

```
            mode = "ASYNC RESET";
```

```
        else if (up)
```

```
            mode = "UP COUNT";
```

```
        else
```

```
            mode = "DOWN COUNT";
```

```
        if (count !== expected_count) begin
```

```
            $display("FAIL: Test %0d | %s | Expected=%b | Got=%b",
```

```

        test_count, mode, expected_count, count);

    failure_count = failure_count + 1;

end else begin

    $display("PASS: Test %0d | %s | Count=%0b",
        test_count, mode, count);

end

end

endtask

initial begin

    $dumpfile("4_bit_up_down_asynchronous_reset_tb.vcd");
    $dumpvars(0, up_down_counter_asynchronous_reset_tb);

    clk = 1'b0;
    reset = 1'b0;
    up = 1'b1;
    test_count = 0;
    failure_count = 0;

    $display("=====");
    $display("Asynchronous Reset 4-bit Up/Down Counter Test  ");
    $display("=====");

    #2 reset = 1'b1;
    check_count(4'd0);
    up = 1'b0;
    #2 reset = 1'b1;
    check_count(4'd0);
    reset = 1'b0;

```

```

up = 1'b1;

$display("\nTesting all 16 UP COUNT states...");
for (value = 1; value <= 16; value = value + 1) begin
    @(posedge clk);
    check_count(value[3:0]);
end

up = 1'b0;

$display("\nTesting all 16 DOWN COUNT states...");
for (value = 15; value >= 0; value = value - 1) begin
    @(posedge clk);
    check_count(value[3:0]);
end

if (failure_count == 0)
    $display("\nPASS: %0d/%0d asynchronous-reset tests passed.",
        test_count, test_count);
else
    $fatal(1, "FAIL: %0d of %0d tests failed.",
        failure_count, test_count);

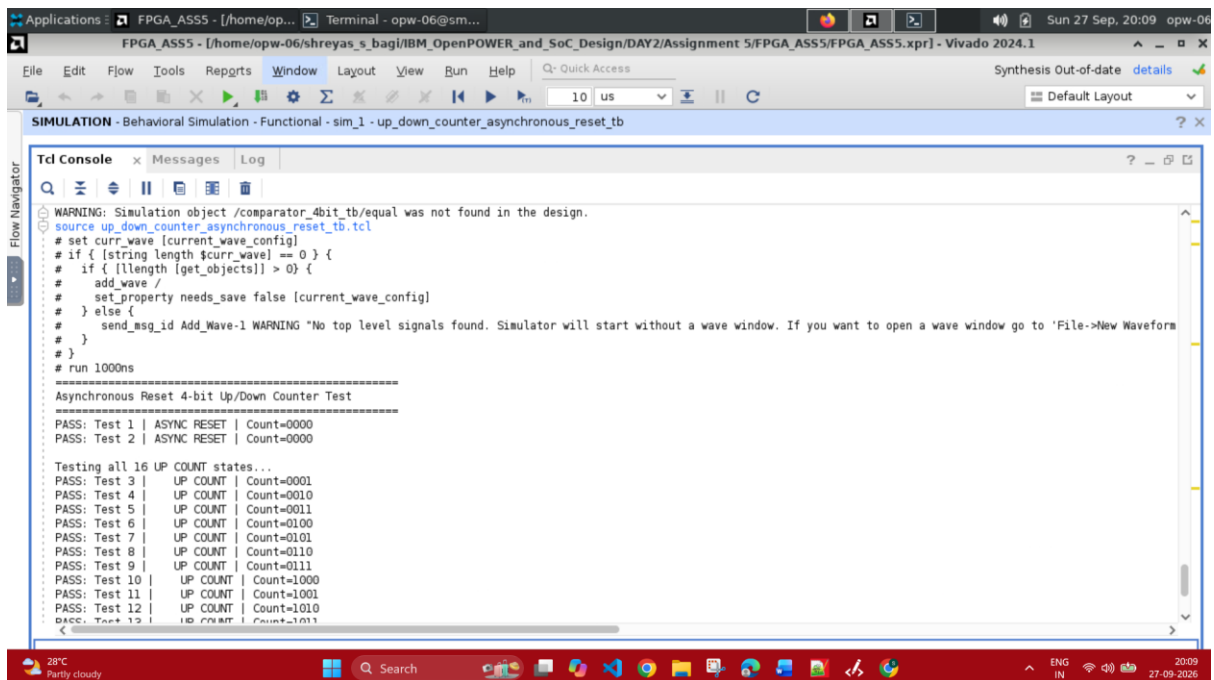
$finish;

end

endmodule

```

Compilation and Simulation Output :



Applications: FPGA_ASS5 - [/home/opw-06/shreyas_s_bagi/IBM_OpenPOWER_and_SoC_Design/DAY2/Assignment 5/FPGA_ASS5/FPGA_ASS5.xpr] - Vivado 2024.1

File Edit Flow Tools Reports Window Layout View Run Help Q-Quick Access Synthesis Out-of-date details

10 us

SIMULATION - Behavioral Simulation - Functional - sim_1 - up_down_counter_asynchronous_reset_tb

Td Console x Messages Log

```
WARNING: Simulation object /comparator_4bit_tb/equals was not found in the design.
source up_down_counter_asynchronous_reset_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [length [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform'"
#   }
# }
# run 1000ns

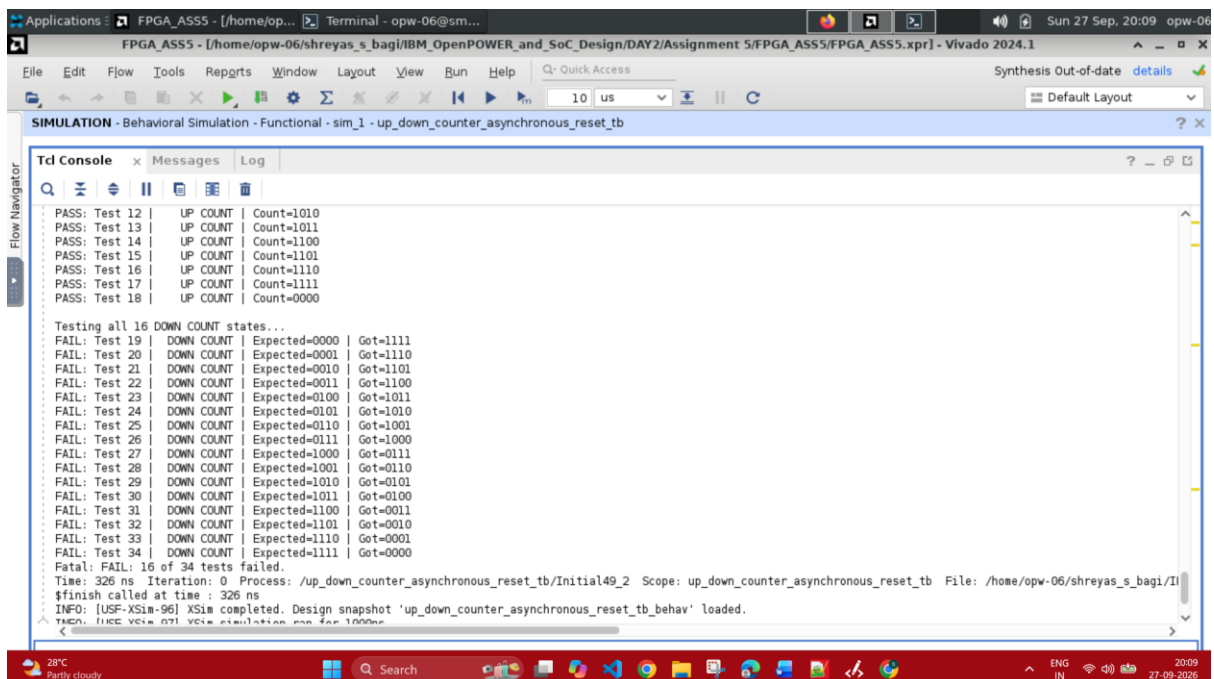
=====
Asynchronous Reset 4-bit Up/Down Counter Test
=====
PASS: Test 1 | ASYNC RESET | Count=0000
PASS: Test 2 | ASYNC RESET | Count=0000

Testing all 16 UP COUNT states...
PASS: Test 3 | UP COUNT | Count=0001
PASS: Test 4 | UP COUNT | Count=0010
PASS: Test 5 | UP COUNT | Count=0011
PASS: Test 6 | UP COUNT | Count=0100
PASS: Test 7 | UP COUNT | Count=0101
PASS: Test 8 | UP COUNT | Count=0110
PASS: Test 9 | UP COUNT | Count=0111
PASS: Test 10 | UP COUNT | Count=1000
PASS: Test 11 | UP COUNT | Count=1001
PASS: Test 12 | UP COUNT | Count=1010
PASS: Test 13 | UP COUNT | Count=1011
```

28°C Partly cloudy

Search

ENG IN 2009 27-09-2026



Applications: FPGA_ASS5 - [/home/opw-06/shreyas_s_bagi/IBM_OpenPOWER_and_SoC_Design/DAY2/Assignment 5/FPGA_ASS5/FPGA_ASS5.xpr] - Vivado 2024.1

File Edit Flow Tools Reports Window Layout View Run Help Q-Quick Access Synthesis Out-of-date details

10 us

SIMULATION - Behavioral Simulation - Functional - sim_1 - up_down_counter_asynchronous_reset_tb

Td Console x Messages Log

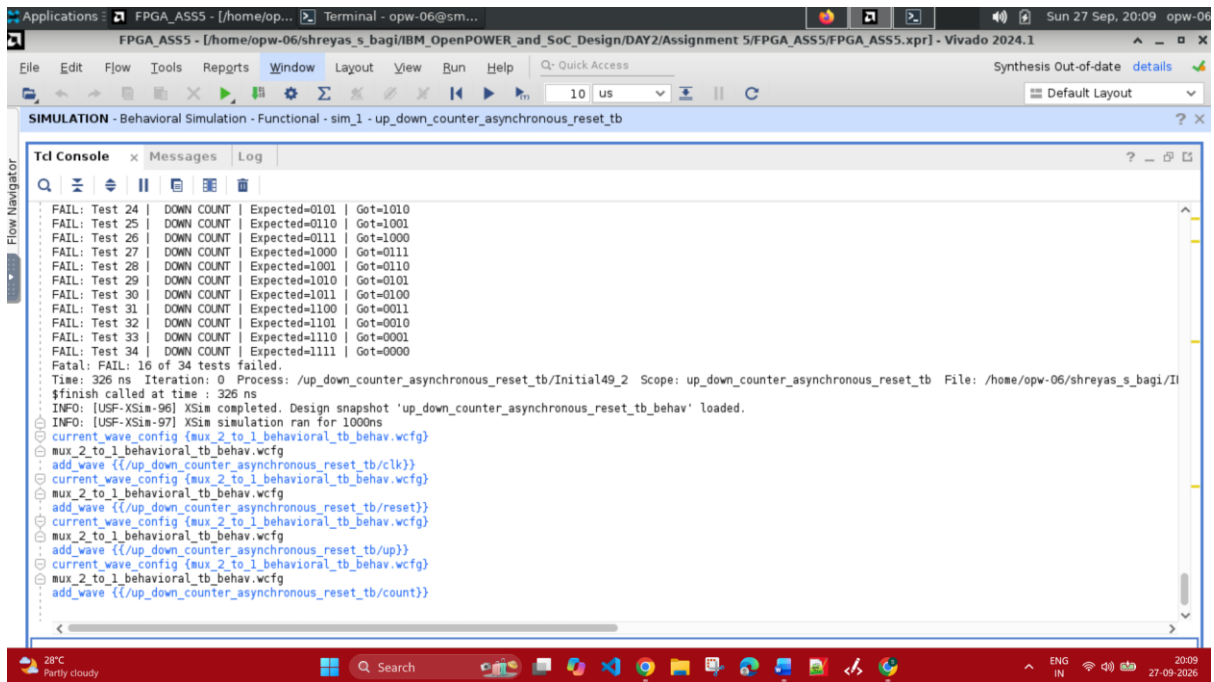
```
PASS: Test 12 | UP COUNT | Count=1010
PASS: Test 13 | UP COUNT | Count=1011
PASS: Test 14 | UP COUNT | Count=1100
PASS: Test 15 | UP COUNT | Count=1101
PASS: Test 16 | UP COUNT | Count=1110
PASS: Test 17 | UP COUNT | Count=1111
PASS: Test 18 | UP COUNT | Count=0000

Testing all 16 DOWN COUNT states...
FAIL: Test 19 | DOWN COUNT | Expected=0000 | Got=1111
FAIL: Test 20 | DOWN COUNT | Expected=0001 | Got=1110
FAIL: Test 21 | DOWN COUNT | Expected=0010 | Got=1101
FAIL: Test 22 | DOWN COUNT | Expected=0011 | Got=1100
FAIL: Test 23 | DOWN COUNT | Expected=0100 | Got=1011
FAIL: Test 24 | DOWN COUNT | Expected=0101 | Got=1010
FAIL: Test 25 | DOWN COUNT | Expected=0110 | Got=1001
FAIL: Test 26 | DOWN COUNT | Expected=0111 | Got=1000
FAIL: Test 27 | DOWN COUNT | Expected=1000 | Got=0111
FAIL: Test 28 | DOWN COUNT | Expected=1001 | Got=0110
FAIL: Test 29 | DOWN COUNT | Expected=1010 | Got=0101
FAIL: Test 30 | DOWN COUNT | Expected=1011 | Got=0100
FAIL: Test 31 | DOWN COUNT | Expected=1100 | Got=0011
FAIL: Test 32 | DOWN COUNT | Expected=1101 | Got=0010
FAIL: Test 33 | DOWN COUNT | Expected=1110 | Got=0001
FAIL: Test 34 | DOWN COUNT | Expected=1111 | Got=0000
Fatal: FAIL: 16 of 34 tests failed.
Time: 326 ns Iteration: 0 Process: /up_down_counter_asynchronous_reset_tb/Initial49_2 Scope: up_down_counter_asynchronous_reset_tb File: /home/opw-06/shreyas_s_bagi/I
$finish called at time: 326 ns
INFO: [USF-XSim-96] XSim completed. Design snapshot 'up_down_counter_asynchronous_reset_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
```

28°C Partly cloudy

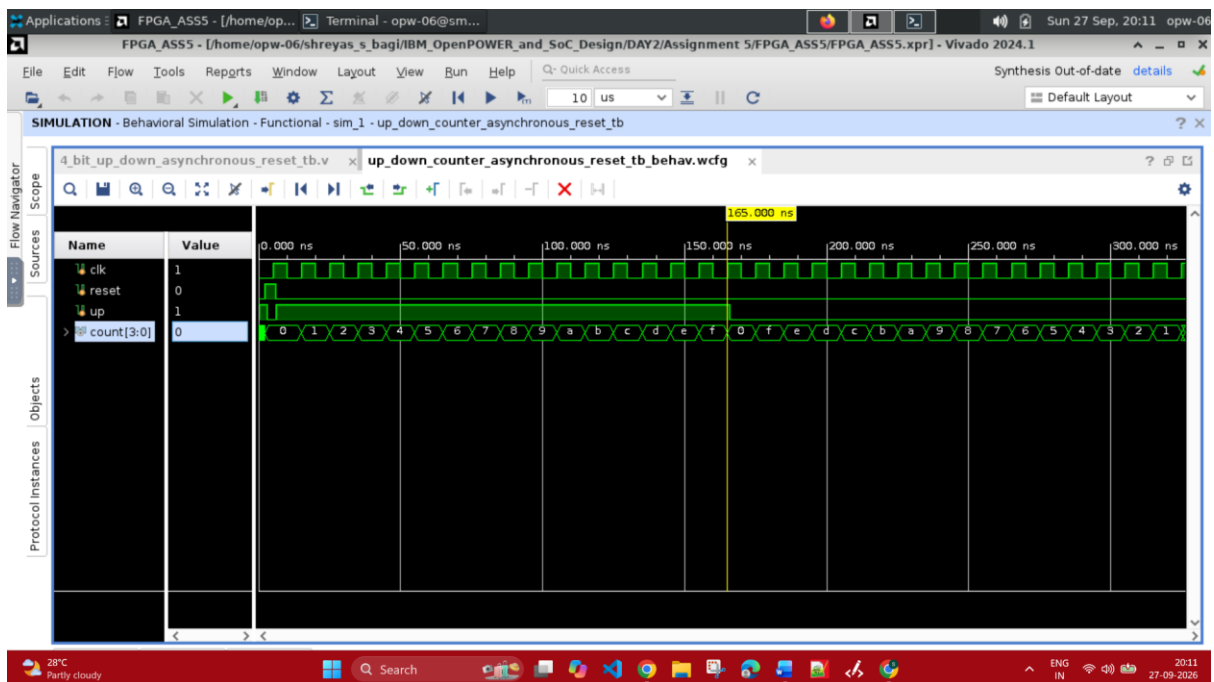
Search

ENG IN 2009 27-09-2026



Simulation Waveform :

The Up Count when up = 1 and Down Count when up = 0.



Asynchronous Reset :

If we observe as at the Time = 2ns. We make the reset = 1 irrespective of the clock edge the changes are taken into effect hence we see count value transition from the Logic X to Logic 0.

